

École Supérieure Polytechnique — ESP

Département Génie Informatique — DGI

Rapport de projet de fin de semestre

Cours : Web Services

Conception et réalisation d'une application de chatroom distribuée fondée sur les protocoles SOAP et REST

*Rapport de projet — architecture polyglotte SOAP et REST
avec frontend hexagonal Port et Adaptateurs*

Programme : Diplôme d'Ingénieur de Conception, 3^e année (DIC3)

Table des matières

Préambule	1
1 Conception	1
1.1 Cahier des charges fonctionnel	1
1.2 Architecture générale	3
1.3 Modèle de données et liaison cohérente	4
1.3.1 Modèle conceptuel	4
1.3.2 Absence de clé étrangère inter-bases	5
1.4 Pattern Port et Adaptateurs côté frontend	6
2 Implémentation	6
2.1 Stack technique	7
2.2 Phase SOAP	7
2.2.1 Service d'authentification Java	7
2.2.2 Service de chat Node.js	8
2.3 Phase REST	8
2.3.1 Service d'authentification Java Jersey	8
2.3.2 Service de chat Python Flask	8
2.4 Frontend PHP	9
2.5 Séquences d'exécution	9
3 Tests et validation	10
3.1 Tests REST avec Postman	11
3.1.1 Organisation de la collection	11
3.1.2 Captures représentatives	11
3.2 Tests SOAP avec SoapUI	12
3.2.1 Organisation du projet	12
3.2.2 Captures représentatives	12

3.3	Démonstration d'interopérabilité et bilan	13
Bibliographie		14

Table des figures

1	Diagramme de cas d'utilisation de la chatroom.	2
2	Architecture globale avec les deux phases coexistantes.	3
3	Modèle conceptuel de données : deux bases isolées par backend.	4
4	Liaison cohérente entre les deux bases via appel inter-service.	5
5	Pattern Port et Adaptateurs côté frontend PHP.	6
6	Séquence d'un envoi de message en phase REST.	9
7	Séquence du polling de messages côté navigateur (1,5 s).	10
8	Exécution complète de la collection Postman.	11
9	Négociation de contenu sur l'endpoint GET /rooms.	11
10	Exécution complète des suites de tests dans SoapUI.	12
11	Trafic SOAP brut pour l'opération login.	13

Liste des tableaux

1	Versions des composants techniques retenus.	7
---	---	---

Préambule

Contexte du projet

Ce rapport accompagne le projet de fin de semestre du cours *Web Services* de la troisième année du diplôme d'ingénieur de conception (DIC3) au département Génie Informatique de l'École Supérieure Polytechnique. Le projet, nommé `FINAL\_PROJECT`, prend la forme d'une chatroom distribuée mettant en oeuvre les deux paradigmes majeurs de services Web : SOAP et REST. La spécificité de notre réalisation tient à la confrontation directe de ces deux paradigmes sur un cas métier identique, complétée par une polyglossie technologique délibérée et par l'application du pattern *Port et Adaptateurs* côté frontend.

Objectifs

Le projet poursuit quatre objectifs pédagogiques. Premièrement, mettre en oeuvre les deux protocoles sur le même scénario fonctionnel afin de comparer concrètement leurs propriétés. Deuxièmement, composer deux services autonomes par phase qui communiquent entre eux via le protocole de la phase, en évitant tout couplage par la base de données. Troisièmement, démontrer l'interopérabilité réelle en faisant cohabiter trois langages : Java, Node.js et Python. Quatrièmement, appliquer le pattern hexagonal sur le frontend PHP afin de basculer entre les deux backends par simple changement d'une variable d'environnement.

Plan du rapport

Le présent document s'organise en trois chapitres. Le premier expose la conception, depuis le cahier des charges fonctionnel jusqu'au modèle de données, en mettant l'accent sur le mécanisme de liaison cohérente entre les deux bases d'un même backend. Le deuxième détaille la mise en oeuvre par service, accompagnée des séquences d'exécution les plus représentatives. Le troisième présente la campagne de validation menée avec Postman pour la phase REST et SoapUI pour la phase SOAP, illustrée par des captures d'écran des échanges réels.

1 Conception

Nous présentons dans ce chapitre la conception du projet, en partant du cahier des charges fonctionnel pour aboutir au modèle de données et à l'organisation hexagonale du frontend. L'accent est mis sur le mécanisme qui garantit la cohérence des données entre les deux bases d'un même backend, sans cle étrangère physique.

1.1 Cahier des charges fonctionnel

L'application réalisée est une chatroom textuelle multi-utilisateurs. Tout visiteur peut créer un compte, se connecter, rejoindre un salon de discussion existant, en créer de nouveaux, envoyer des messages et lire ceux des autres participants. Deux services autonomes prennent en charge le métier : *Auth-*

Service expose les opérations `register`, `login`, `logout` et `validateToken` ; *ChatService* expose `createRoom`, `listRooms`, `sendMessage` et `getMessages`.

La figure 1 synthétise ces interactions. Le diagramme distingue deux acteurs, le *Visiteur* anonyme et l'*Utilisateur authentifié*, et met en évidence par des relations «include» la sollicitation systématique de `validateToken` par chaque cas du *ChatService*.

Diagramme de cas d'utilisation — Chatroom

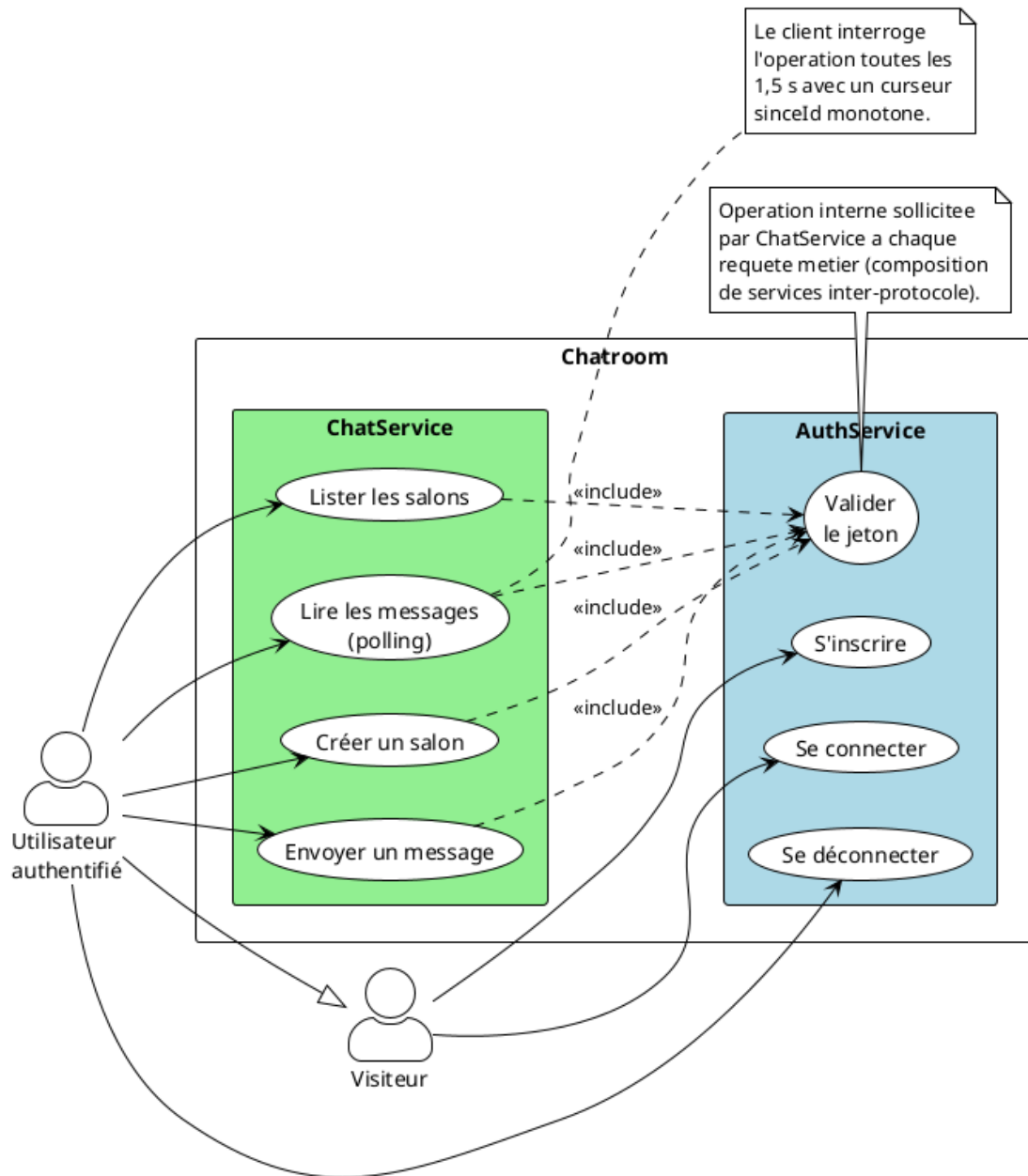


FIGURE 1 – Diagramme de cas d'utilisation de la chatroom.

1.2 Architecture générale

Notre architecture comporte trois couches : un frontend PHP de présentation, deux services backend par phase, et une instance PostgreSQL pour la persistance. Les deux phases SOAP et REST coexistent dans une même topologie et sont sélectionnées par le frontend via la variable d'environnement `BACKEND_MODE`. La figure 2 en donne la vue d'ensemble.

Architecture globale du système

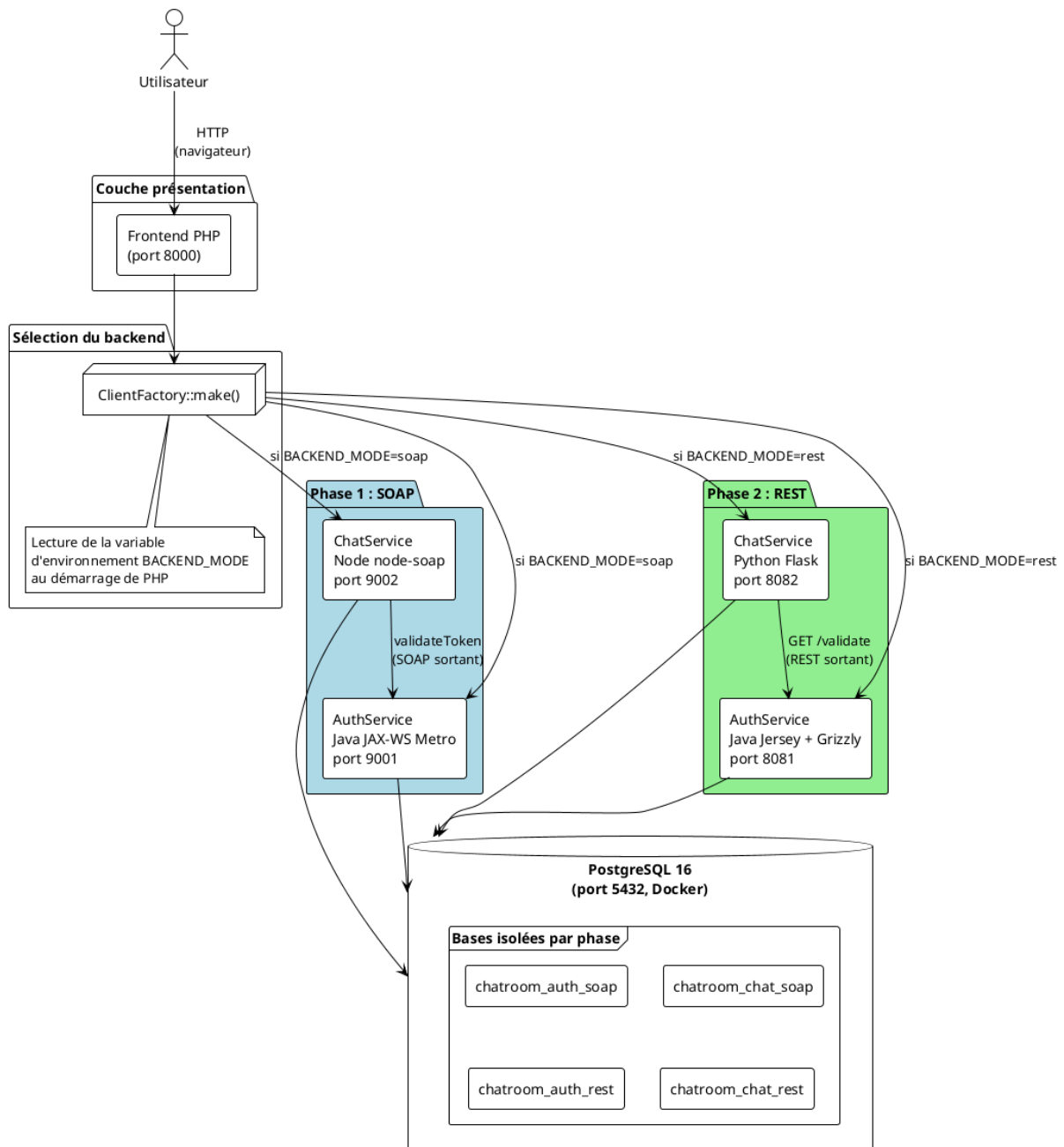


FIGURE 2 – Architecture globale avec les deux phases coexistantes.

1.3 Modèle de données et liaison cohérente

1.3.1 Modèle conceptuel

Le modèle conceptuel comprend quatre entités : USER et SESSION pour la sphère d'authentification, ROOM et MESSAGE pour la sphère de discussion. La figure 3 les présente. Chaque sphère réside dans une base de données distincte : chatroom_auth_* pour l'authentification et chatroom_chat_* pour les messages, avec deux jeux complets pour les phases SOAP et REST.

Modèle Conceptuel de Données (MCD) — backends isolés

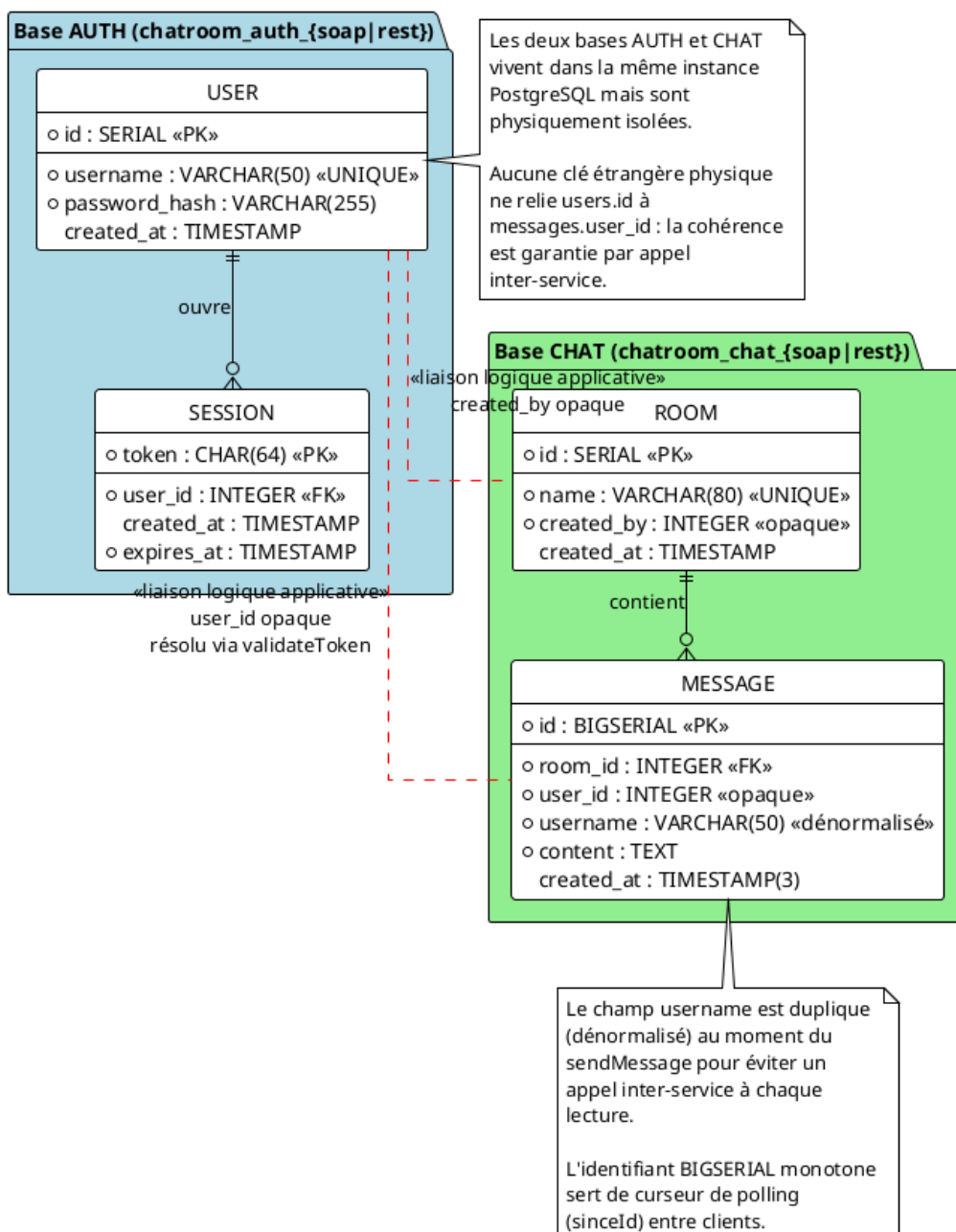


FIGURE 3 – Modèle conceptuel de données : deux bases isolées par backend.

L'identifiant des messages est de type BIGSERIAL : sa croissance monotone permet au navigateur d'utiliser un curseur `sinceId` pour ne recevoir que les nouveaux messages lors du polling. Le champ `username` est dénormalisé dans la table `messages` pour épargner un appel inter-service à chaque lecture d'historique.

1.3.2 Absence de clé étrangère inter-bases

PostgreSQL n'autorise pas de contrainte de clé étrangère entre deux bases distinctes. Le champ `messages.user_id` ne peut donc pas référencer physiquement `users.id`. Sans précaution, cette absence ouvrirait la porte à des incohérences référentielles. Notre solution consiste à déplacer la garantie de cohérence du niveau base de données vers le niveau applicatif, en exploitant la composition de services.

Avant toute insertion dans la table `messages`, le `ChatService` interroge le `AuthService` pour résoudre le couple (`user_id`, `username`) à partir du jeton fourni. Si le jeton est invalide ou expiré, l'insertion n'a pas lieu. Cette validation préalable joue le rôle qu'aurait joué une contrainte de clé étrangère si les deux tables coexistaient. La figure 4 retrace cette séquence.

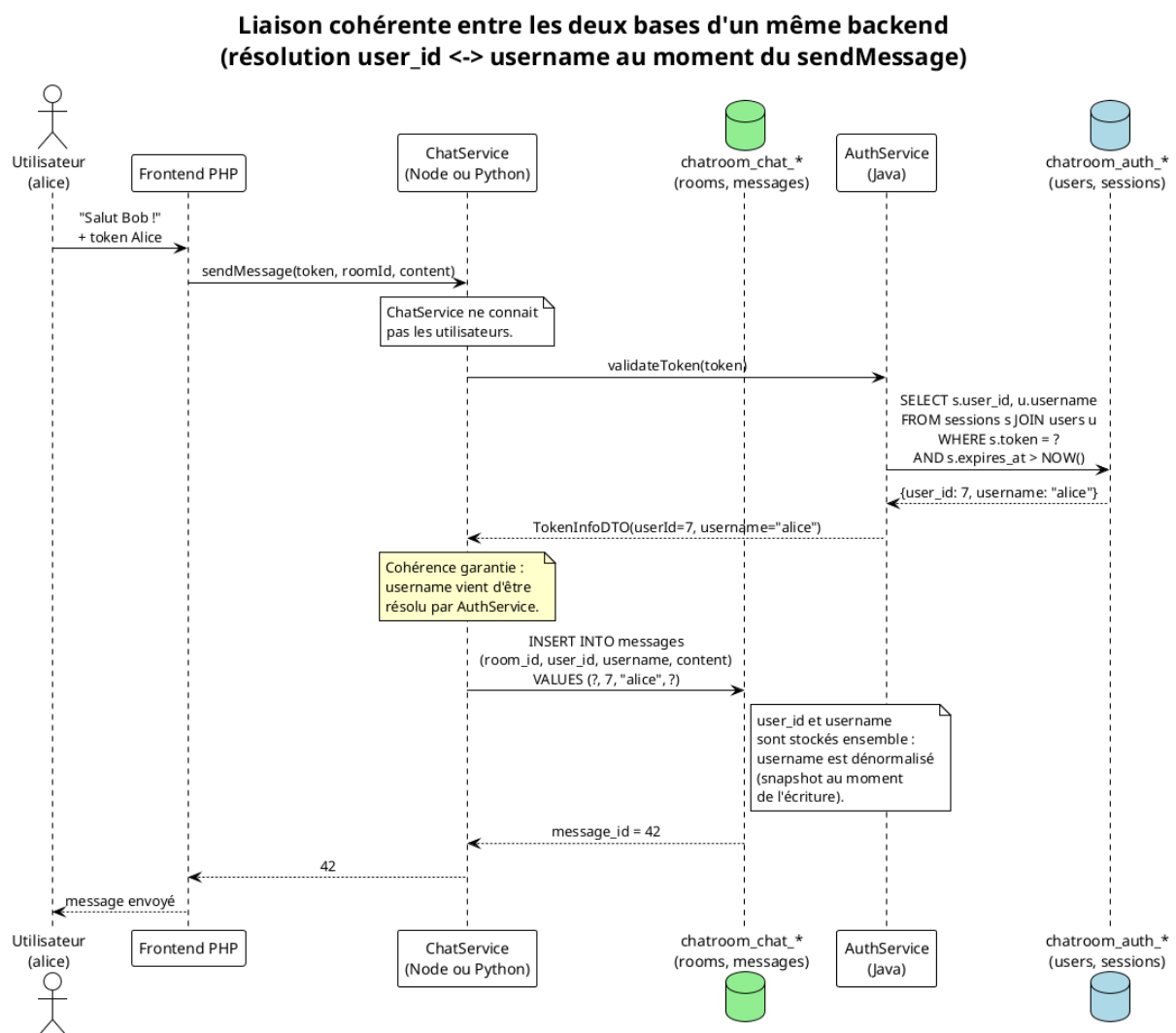


FIGURE 4 – Liaison cohérente entre les deux bases via appel inter-service.

Ce dispositif transforme une *contrainte de référence physique* en une *contrainte de référence applicative*, médiatisée par un protocole de service Web. La cohérence est sacrifiée au niveau du stockage mais rétablie au niveau du domaine par une orchestration explicite. Cette transposition est centrale dans les architectures de microservices.

1.4 Pattern Port et Adapteurs côté frontend

Le frontend PHP n'a aucune raison légitime de connaître le protocole technique de ses services backend. Le pattern *hexagonal* permet de matérialiser cette ignorance. La figure 5 montre l'organisation des classes : l'interface `ClientInterface` déclare les huit opérations métier ; deux adaptateurs concrets l'implémentent, `SoapBackendClient` encapsulant deux `SoapClient` PHP et `RestBackendClient` encapsulant des appels `cURL` ; la fabrique `ClientFactory::make()` sélectionne l'adaptateur au démarrage selon `BACKEND_MODE`.

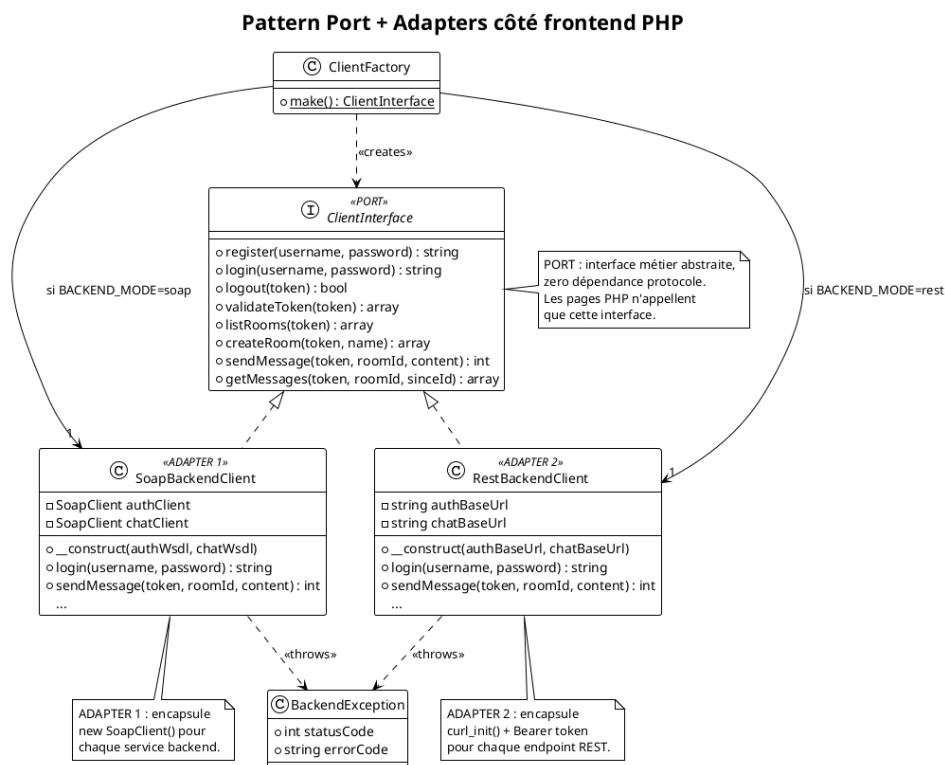


FIGURE 5 – Pattern Port et Adapteurs côté frontend PHP.

Aucune ligne de code des pages applicatives ne distingue SOAP de REST. La bascule entre les deux phases passe exclusivement par la fabrique, ce qui valide concrètement le principe d'inversion de dépendance porté par le pattern.

2 Implémentation

Ce chapitre détaille la mise en oeuvre du projet. Nous présentons d'abord la stack retenue, puis l'implémentation des services par paradigme, et enfin le frontend qui consomme indifféremment les deux phases.

2.1 Stack technique

Le tableau 1 résume les composants techniques retenus, avec leur version exacte. Le choix d'unifier le langage Java pour les deux services d'authentification réduit la dispersion sur la fonction la plus critique, tandis que la diversité Node.js / Python pour les services de chat introduit la polyglossie souhaitée.

TABLEAU 1 – Versions des composants techniques retenus.

Composant	Rôle	Version
PostgreSQL	SGBD relationnel (Docker)	16
Java JDK	AuthService SOAP et REST	21
JAX-WS Metro	SOAP côté Java	4.0.2
Jersey + Grizzly	REST côté Java	3.1.8
Node.js / node-soap	ChatService SOAP	20 / 1.1.6
Python / Flask	ChatService REST	3.12 / 3.0.3
PHP	Frontend	8.2+
jBCrypt	Hachage des mots de passe (cost 10)	0.4

2.2 Phase SOAP

2.2.1 Service d'authentification Java

Le service d'authentification SOAP est écrit selon l'approche *code-first*. Une interface annotée `@WebService` déclare les opérations exposées et Metro génère automatiquement le contrat WSDL au démarrage. Le Listing 1 en montre l'extrait essentiel.

```
1 @WebService(targetNamespace = "http://chatroom.dic3/auth", name = "
   AuthService")
2 public interface AuthService {
3     @WebMethod
4     String login(@WebParam(name = "username") String username ,
5                 @WebParam(name = "password") String password) throws
6                 AuthFault;
7
8     @WebMethod
9     TokenInfoDTO validateToken(@WebParam(name = "token") String token)
10    throws AuthFault;
11    // register et logout omis
12 }
```

Listing 1 – Interface du service d'authentification SOAP.

L'implémentation s'appuie sur jBCrypt pour le hachage des mots de passe et sur `SecureRandom` pour générer des jetons hexadécimaux de soixante-quatre caractères stockés dans la table `sessions` avec un TTL de huit heures.

2.2.2 Service de chat Node.js

Le service de chat suit l'approche inverse : *contract-first*. Le WSDL est rédigé à la main, puis node-soap implémente le service à partir de ce contrat. Le point notable de cette implémentation réside dans l'appel SOAP sortant qui valide chaque jeton auprès du service Java, illustré par le Listing 2.

```
1 async function validateToken(token) {  
2   const c = await soap.createClientAsync(process.env.AUTH_WSDL_URL);  
3   const [resp] = await c.validateTokenAsync({ token });  
4   return { userId: resp.return.userId, username: resp.return.username };  
5 }
```

Listing 2 – Appel SOAP sortant de ChatService Node vers AuthService Java.

Cette validation s'exécute avant chaque insertion dans la table messages et matérialise la liaison cohérente présentée au chapitre précédent.

2.3 Phase REST

2.3.1 Service d'authentification Java Jersey

L'équivalent REST repose sur Jersey avec serveur embarqué Grizzly. La négociation de contenu, déclarative, autorise le service à répondre indifféremment en JSON ou en XML selon l'en-tête Accept du client. Le Listing 3 expose la ressource.

```
1 @Path("/")  
2 @Produces({ MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML })  
3 @Consumes({ MediaType.APPLICATION_JSON, MediaType.APPLICATION_XML })  
4 public class AuthResource {  
5   @POST @Path("/login")  
6   public TokenDTO login(CredentialsDTO body) { ... }  
7  
8   @GET @Path("/validate")  
9   public TokenInfoDTO validate(@HeaderParam("Authorization") String  
10     authorization) { ... }  
11 }
```

Listing 3 – Ressource REST avec négociation de contenu.

JAXB sérialise automatiquement les DTO annotés `@XmlRootElement` dans le format adapté.

2.3.2 Service de chat Python Flask

Le service Python est structuré en *blueprints* Flask. L'authentification est portée par un décorateur `@require_auth` apposé sur chaque route protégée. Ce décorateur extrait le jeton de l'en-tête Authorization, délègue sa validation au service Java par un appel HTTP, et place le résultat dans l'objet `g.user`. La négociation de contenu est ici explicite : la fonction `render` inspecte l'objet `request.accept_mimetypes` et choisit JSON par défaut ou XML via `xmltodict`.

2.4 Frontend PHP

Les huit opérations métier sont déclarées par l'interface `ClientInterface`. L'adaptateur `SoapBackendClient` encapsule deux instances de `SoapClient` ; chaque méthode du port se traduit par un appel SOAP, et les `SoapFault` sont reconvertis en `BackendException`. L'adaptateur `RestBackendClient` suit la même logique avec `cURL` et un en-tête `Authorization: Bearer`. La fabrique `ClientFactory::make()` lit la variable `BACKEND_MODE` et instancie l'adaptateur approprié.

2.5 Séquences d'exécution

La figure 6 illustre le flux complet d'un envoi de message en phase REST, depuis le navigateur jusqu'à l'insertion en base, en passant par le middleware d'authentification et l'appel inter-service vers Jersey. La composition de services y est observable directement.

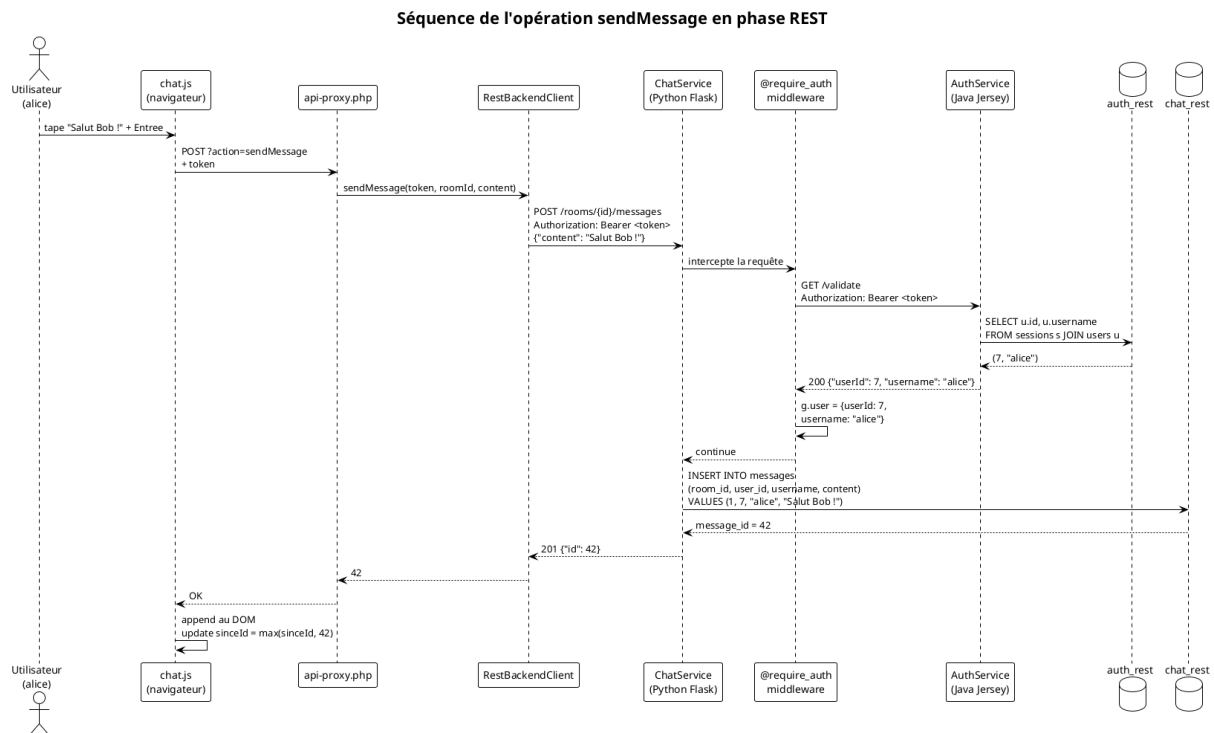


FIGURE 6 – Séquence d'un envoi de message en phase REST.

La livraison des messages au navigateur s'appuie sur une scrutation périodique, illustrée par la figure 7. La variable locale `sinceId` sert de curseur monotone : chaque appel ne récupère que les messages dont l'identifiant est strictement supérieur à ce curseur, ce qui rend les itérations incrémentales et insensibles aux décalages d'horloge entre serveurs.

Séquence du polling côté navigateur (toutes les 1.5 secondes)

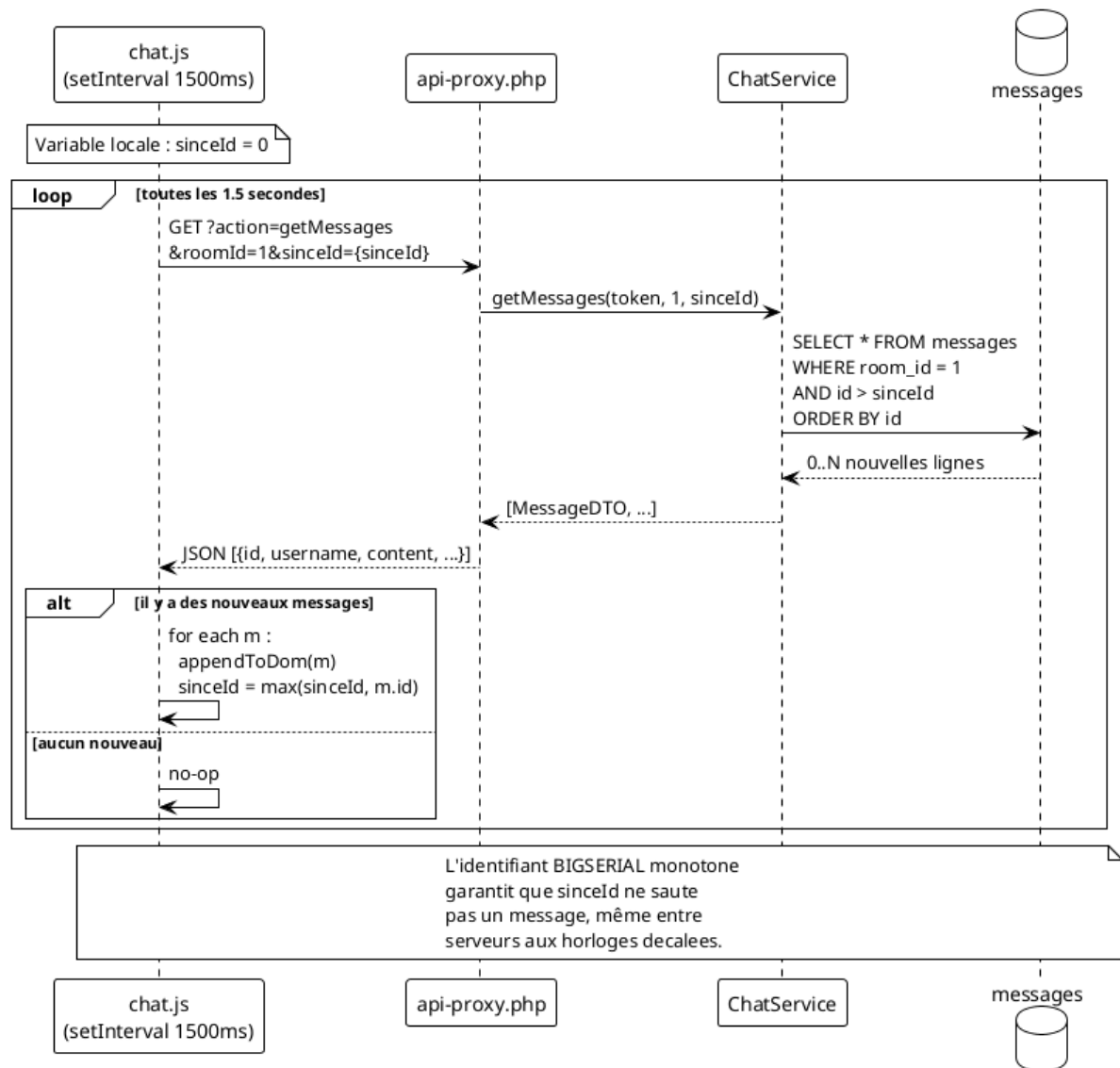


FIGURE 7 – Séquence du polling de messages côté navigateur (1,5 s).

3 Tests et validation

Nous rendons compte ici de la campagne de validation menée sur les deux phases. Postman a servi à tester l'API REST, SoapUI à éprouver les services SOAP. Chaque outil a été mobilisé selon deux angles : exécution unitaire de chaque opération et exécution chaînée d'un scénario complet reproduisant un parcours utilisateur réaliste.

3.1 Tests REST avec Postman

3.1.1 Organisation de la collection

La collection `docs/postman/chatroom-rest.postman\_collection.json` est organisée par service. Un dossier rassemble les opérations de *AuthService*, un autre celles de *ChatService*. L'environnement `chatroom-rest.postman\_environment.json` déclare douze variables : les URL des services, les comptes de test, et des variables dynamiques (tokens, identifiant de salon, curseur de scrutation) remplies automatiquement par les scripts de test après chaque réponse.

3.1.2 Captures représentatives

La figure 8 présente le résultat de l'exécution complète dans le *Collection Runner*. Toutes les assertions sont satisfaites, confirmant la cohérence du parcours utilisateur de bout en bout.

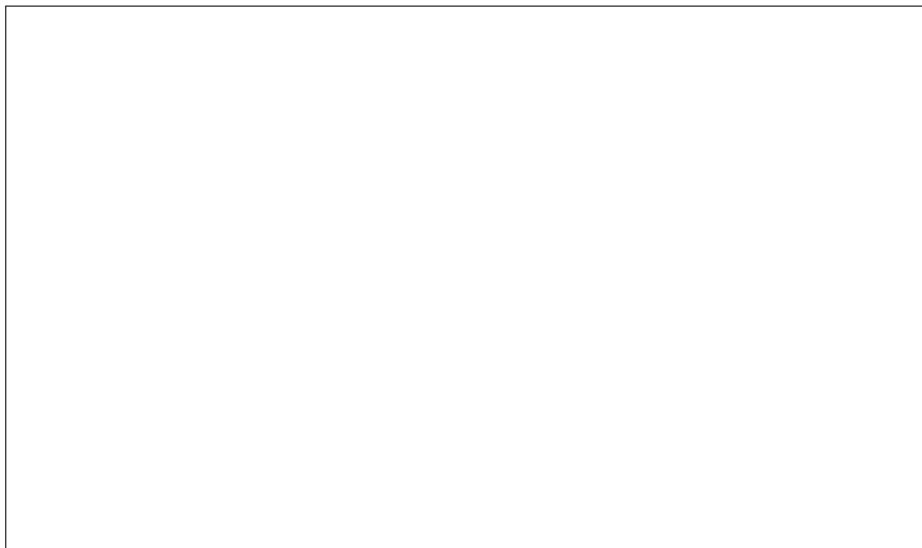
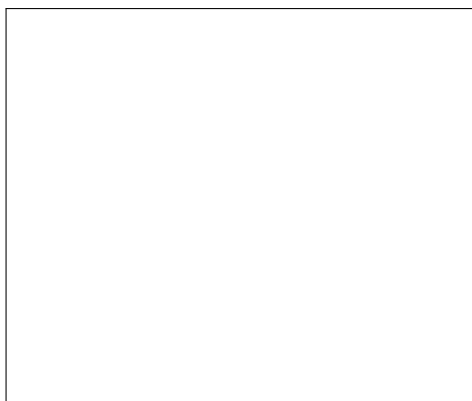
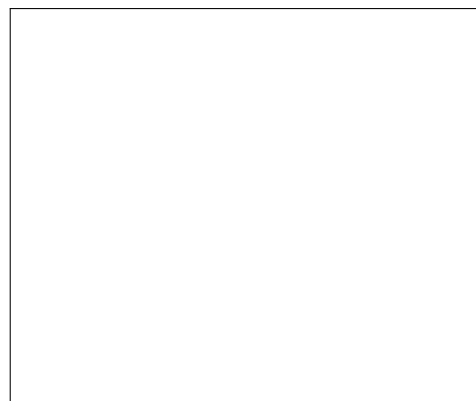


FIGURE 8 – Exécution complète de la collection Postman.

Les figures figure 9a et figure 9b illustrent la négociation de contenu : un même endpoint sert le format JSON ou le format XML selon la valeur de l'en-tête `Accept`.



(a) Format JSON.



(b) Format XML.

FIGURE 9 – Négociation de contenu sur l'endpoint `GET /rooms`.

3.2 Tests SOAP avec SoapUI

3.2.1 Organisation du projet

Le projet `docs/soapui/chatroom-soap-soapui-project.xml` comprend deux interfaces correspondant aux services SOAP, ainsi que trois suites de tests : une pour *AuthService*, une pour *ChatService* et une dernière pour la déconnexion finale. Le chaînage entre les étapes du scénario repose sur six *Property Transfers* qui extraient les jetons, l'identifiant de salon créé, l'identifiant du dernier message et le curseur de scrutation.

3.2.2 Captures représentatives

La figure 10 montre le résultat de l'exécution complète. Toutes les barres apparaissent en vert, signe que les vingt-trois assertions définies sont satisfaites.

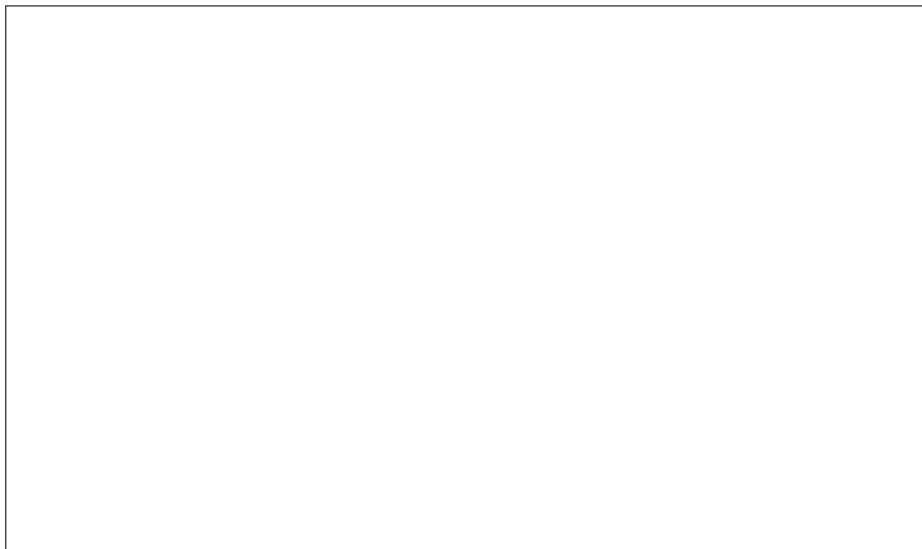
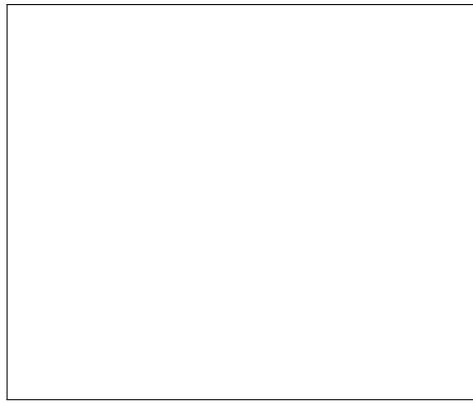
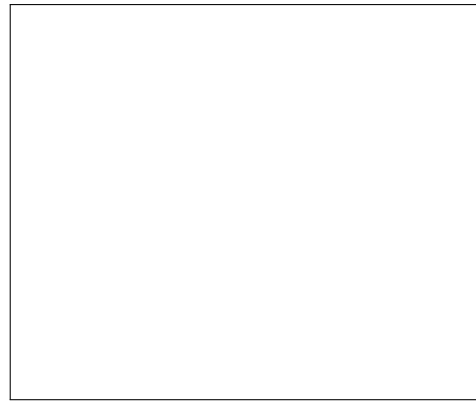


FIGURE 10 – Exécution complète des suites de tests dans SoapUI.

L'un des grands intérêts de SoapUI est sa capacité à exposer le trafic SOAP brut. La figure 11 présente l'enveloppe d'une requête `login` et celle de sa réponse, avec leurs en-têtes HTTP. Cette transparence facilite l'analyse et confirme la conformité aux contrats WSDL.



(a) Enveloppe de la requête.



(b) Enveloppe de la réponse.

FIGURE 11 – Trafic SOAP brut pour l'opération `login`.

3.3 Démonstration d'interopérabilité et bilan

Le test le plus probant de notre architecture consiste à interrompre volontairement le service d'authentification pendant que le service de chat reste actif. Le frontend continue d'afficher les messages déjà reçus, mais toute tentative d'envoi échoue avec une erreur d'authentification propre. Cette propagation contrôlée de l'erreur valide la composition de services : *ChatService* consulte systématiquement son pair avant chaque opération sensible et ne fait pas confiance à un jeton non vérifié.

La bascule entre les deux backends, opérée par simple changement de la variable `BACKEND_MODE` suivi d'un redémarrage du serveur PHP, complète cette validation : aucune ligne de code applicatif n'a été modifiée, et l'application offre la même expérience utilisateur quel que soit le protocole sous-jacent. Cette double démonstration confirme l'atteinte des objectifs énoncés en préambule : composition de services observable, polyglossie effective, pattern hexagonal éprouvé.

Références